# 01_CODE_ALONG_LAB

**Pintop | 2026 Training**

# CLASSROOM CODE-ALONG - Migrations, Transactions & Query Optimisation (Backend Design & Engineering)

**Pintop Technologies | Class 06 | Instructor and student companion**

## What we will produce

Recreate a documented excerpt from the **real project** (`database/migrations/2026_01_01_000002_paytrack.php`), practise a small safe variation, verify using the complete app or appropriate tool, and record both success and failure evidence. Preserve the original project.

**Why:** You add a column to the database on your machine. Your colleague's machine does not have it, so their code breaks. Today we build and change the database in a way everyone can repeat, and make sure a multi-step change either fully succeeds or not at all.

## 1. Find and read the real file

`codebase/database/migrations/2026_01_01_000002_paytrack.php`

The excerpt is in `REFERENCE EXCERPT.php`. Its job is to give you something to type, trace and understand. It is not guaranteed to run on its own: it is part of a larger file. Do not overwrite project source with a fragment.

## 2. Set up the real project

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

## 3. Type in stages

### Stage 1

**Say:** "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
 public function up():void {
```

### Stage 2

**Say:** "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
  Schema::create('wallets',function(Blueprint $t){$t->uuid('id')->primary();$t->foreignId('user_id')->const
  Schema::create('ledger_entries',function(Blueprint $t){$t->id();$t->uuid('wallet_id');$t->string('directi
  Schema::create('transfer_requests',function(Blueprint $t){$t->id();$t->foreignId('user_id')->constrained(
```

### Stage 3

**Say:** "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
        }
        public function down():void {Schema::dropIfExists('transfer_requests');Schema::dropIfExists('ledger_entrie
       };
```

## 4. Complete these activities

1. **Locate the table, query or migration in the referenced file, then sketch the before-state.**
2. **On a disposable local database, adjust one field, query or fixture and predict what must change.**
3. **Run the local check, verify the resulting rows and explain rollback or restoration.**

## 5. Check, debug and record

```
php artisan route:list --path=api
php artisan test
```

**Expected check:** If the local app starts or its tests pass, capture evidence of the specific behaviour tied to this class, not merely an application startup. Test one failed or edge input too. Log the exact error if blocked.

## 6. Marking

- 4/20: explain concept and input/output in plain language
- 4/20: accurate typed or edited work
- 4/20: local proof and explanation
- 4/20: safe variation and wrong-case recovery
- 4/20: short documented submission

**Lab ethics:** no unapproved public scanning, no third-party tokens and no real banking customer data.